

μOS++ Test Framework: xpack- development vs smp

**Comparative Architecture Analysis, Harness Topologies, and
Transition Pathways**

Comparative Analysis: μ OS++ IIIe Test Framework

- 1. Executive Summary & Paradigm Shift
- 2. Core Architecture & Library Under Test Selection
- 3. Test Topology: Harness Suites vs. Port Board Tests
- 4. Platform Coverage & Matrix Expansion
- 5. Execution Harness, Runners & Verdict Protocol (test_smpl/)
- 6. Manifest & Configuration Matrix (tests/package.json)
- 7. Device Emulation Plumbing (tests/device-qemu-cortexm/)
- 8. Summary Comparison Matrix
- 9. Transition Pathways: Rebuilding the SMP Test Framework from xpack-development

Comparative Analysis: `μOS++` IIIe Test Framework

`xpack-development` (Standard Single-Core) vs. `smp` (Multi-Core Unified Harness)

1. Executive Summary & Paradigm Shift

The testing strategy of `μOS++` IIIe revolves around a core tenet: **write test sources once, then compile and run them across as many toolchains and target platforms as possible**. However, the architectural realization of this philosophy diverges fundamentally between the `xpack-development` branch and the `smp` branch.

Dimension	<code>xpack-development</code> (Standard)	<code>smp</code> Branch (Unified SMP)
Repository Topology	Monolithic single-repository: The test framework compiles and links against the local in-tree kernel (<code>..</code>).	Multi-repository unified workspace: The test harness acts as an orchestrator across sibling architecture ports (<code>aarch32</code> , <code>aarch64</code> , <code>cortexm</code> , <code>posix-arch</code>).
Target Scheduling	Single-core only (<code>OS_NCPU = 1</code> or non-SMP).	Heterogeneous multi-core SMP (<code>OS_NCPU</code> 1 to 4) alongside legacy single-core backwards compatibility.
Library Under Test	Static target <code>micro-os-plus::iii</code> encompassing kernel, posix-io, startup, and drivers.	Dynamically selected architecture port (e.g., <code>micro-os-plus::aarch32</code> , <code>micro-os-plus::cortexm</code>) which transitively pulls modular kernel targets.
Test Categorization	All tests are harness-owned portable suites located in <code>tests/sources/</code> .	Clear two-tier hierarchy: Harness Suites (portable, entry <code>os_main()</code>) and Port Board Tests (silicon-specific, entry <code>main()</code>).
Execution & Runners	Direct binary invocation and basic QEMU execution without runner abstraction.	Dedicated runner layer (<code>test_smpl/</code>) with execution wrappers (<code>run-qemu.sh</code> , <code>run-host.sh</code> , <code>run-hw.sh</code>).
Verdict Protocol	Process exit code and semihosting return code (<code>0</code> = pass).	Dual-channel: machine-greppable <code>RESULT: PASS FAIL SKIP</code> string parsing plus semihosted <code>SYS_EXIT</code> termination (<code>hw_result</code>).
Platform Count	9 platforms (1 host, 4 QEMU Cortex-M, 4 physical boards).	22 platforms (expanded to include dual M33, AArch32/64 Pis, Lyra, RP2350).

2. Core Architecture & Library Under Test Selection

2.1 Kernel Decomposition in Root `CMakeLists.txt`

On `xpack-development`, the root `CMakeLists.txt` exports a single interface library `micro-os-plus::iii` that compiles all source files together: kernel scheduler, POSIX I/O, device drivers, startup, semihosting, and trace diagnostics.

On `smp`, the kernel is decoupled into granular interface libraries: * `micro-os-plus::iii` (and alias `micro-os-plus::iii-core`): Contains strictly the core RTOS kernel primitives (`src/rtos/*`, `src/utils/lists.cpp`). * `micro-os-plus::iii-posix-io`: Decoupled POSIX file descriptors, file systems, and sockets. * `micro-os-plus::iii-startup`: Exception handlers and hardware reset/initialization hooks. * `micro-os-plus::iii-semihosting`: C syscall wrappers over ARM semihosting. * `micro-os-plus::port-smp-decls`: Shared C++ port contract headers (`port/smp-common`). * `micro-os-plus::test-support`: Test scaffolding headers (`hw_result.hpp`) and compile-time contract checks (`board-contract.cpp`).

2.2 Dynamic Port Dispatch in `tests/cmake/tests-main.cmake`

The orchestration script `tests-main.cmake` undergoes a fundamental transformation:

- `xpack-development`: Always executes: `cmake add_subdirectory (".." "top-bin")`
- `smp`: Selects the library under test dynamically based on the prefix of `PLATFORM_NAME`:

```
cmake if (PLATFORM_NAME MATCHES "^aarch32") add_subdirectory ("${UOS_AARCH32_DIR}" "port-bin")  
elseif (PLATFORM_NAME MATCHES "^aarch64") add_subdirectory ("${UOS_AARCH64_DIR}" "port-bin")  
elseif (PLATFORM_NAME MATCHES "^native") add_subdirectory ("${UOS_POSIX_ARCH_DIR}" "port-bin")  
elseif (PLATFORM_NAME MATCHES "^cortexm" OR PLATFORM_NAME MATCHES "^qemu-cortex")  
add_subdirectory ("${UOS_CORTEXM_DIR}" "port-bin") elseif (PLATFORM_NAME MATCHES  
"^pico2" OR PLATFORM_NAME MATCHES "^2xcortex") add_subdirectory ("${UOS_CORTEXM_DIR}" "port-bin")  
else () # Fallback to plain in-tree kernel for upstream boards add_subdirectory (".." "top-bin") endif ()
```

Because each port imports `micro-os-plus::iii`, dynamically dispatching to the port prevents CMake duplicate target collision errors (`alias micro-os-plus::iii already exists`).

3. Test Topology: Harness Suites vs. Port Board Tests

3.1 Two-Tier Test Classification

Test Tier	Location	Entry Point	Build Mechanism	Execution Scope
Harness Suites	tests/sources/ <name>/	os_main(argc, argv)	Exported as INTERFACE libraries (test:: <name>) in harness; linked with micro-os- plus::platform	Portable across all supported architectures and emulators
Port Board Tests	<port>/test/ <board>/<app>/	main()	Built directly by the architecture port's builder (<port>/ test/ CMakeLists.txt)	Silicon- and board- specific tests exercising hardware peripherals

3.2 Dynamic Board Test Registration

In `xpack-development`, platform `CMakeLists.txt` files hardcode test executable declarations:

```
add_test_executable(rtos-apis-test)
add_test_executable(mutex-stress-test)
add_test_executable(cmsis-os-validator-test)
```

In `smp`, port platforms (such as `tests/platforms/native/CMakeLists.txt` and `tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt`) delegate test discovery to the filesystem:

```
file (GLOB _test_dirs LIST_DIRECTORIES true "${_port_test_dir}/*")
foreach ( _dir IN LISTS _test_dirs)
    # Enumerates each test folder and registers CTest targets dynamically:
    # <PLATFORM_NAME>-<app>-qemu, <PLATFORM_NAME>-<app>-hwd, or <PLATFORM_NAME>-<app>-host
endforeach ( )
```

Adding a test in a port requires zero edits in `tests/platforms/`.

3.3 New and Modified Test Suites

- **New Suite:** `fp-switch` :
 - Located in `tests/sources/fp-switch/src/main.cpp`.
 - Specifically created for SMP validation.
 - Spawns 6 unpinned threads across three priorities. Each thread loads a unique, deterministic bit pattern into FPU registers `s0 – s31` and the `FPSCR` condition flags via inline assembly

(`hold_fp_registers`), spins across timer ticks to provoke preemption, and validates that register contexts are preserved when threads migrate between physical cores (`migrated_rounds`).

- **Enhanced Suite:** `rtos-apis` :
 - Dynamic memory pool size `OS_INTEGER_RTOS_DYNAMIC_MEMORY_SIZE_BYTES` enlarged from **14 KiB** to **512 KiB** to accommodate FatFs working allocations without invoking the fatal out-of-memory hook.
 - FatFs test execution gated behind `#if !defined(OS_EXCLUDE_RTOS_APIS_FATFS)` for bare-metal targets without storage.
- **Refined Suite:** `cmsis-os-validator` :
 - Disabled deprecated `libucontext` on Linux host platforms in favor of native glibc `ucontext`.

4. Platform Coverage & Matrix Expansion

The platform count increases from **9** to **22** under `tests/platforms/` :

```
tests/platforms/
├─ 2xcortex-m33/           [NEW] Dual Cortex-M33 in QEMU (MPS2-AN521, SMP 2)
├─ aarch32-luckfox-lyra/   [NEW] Luckfox Lyra (3× Cortex-A7, hardware only)
├─ aarch32-rpi-zero-2w/    [NEW] Raspberry Pi Zero 2 W (4× Cortex-A53, AArch32)
├─ aarch32-rpi3b/          [NEW] Raspberry Pi 3 B (4× Cortex-A53, AArch32)
├─ aarch64-rpi-zero-2w/    [NEW] Raspberry Pi Zero 2 W (4× Cortex-A53, AArch64)
├─ aarch64-rpi3b/          [NEW] Raspberry Pi 3 B (4× Cortex-A53, AArch64)
├─ cortexm-nucleof411/     [NEW] ST Nucleo-F411RE via Cortex-M port
├─ cortexm-pico2-pizero/    [NEW] RP2350B Pi-Zero format (2× Cortex-M33)
├─ cortexm-pico2-rp2350b-psram/[NEW] RP2350B with 8 MB PSRAM (2× Cortex-M33)
├─ cortexm-pico2/          [NEW] Raspberry Pi Pico 2 (2× Cortex-M33)
├─ cortexm-weactf411/      [NEW] WeAct Studio F411CE via Cortex-M port
├─ cortexm-weactf412/      [NEW] WeAct Studio F412RE via Cortex-M port
├─ native/                 [MOD] POSIX port host process (host threads as CPUs)
├─ nucleo-f411re/          [LEG] Upstream single-core hardware platform
├─ nucleo-f767zi/          [LEG] Upstream single-core hardware platform
├─ nucleo-h743zi/          [LEG] Upstream single-core hardware platform
├─ pico2-1cpu/             [NEW] Generic Cortex-M33 in QEMU (MPS2-AN505, 1 core)
├─ qemu-cortex-m0/         [MOD] Generic M0/M3 in QEMU (MPS2-AN385)
├─ qemu-cortex-m3/         [MOD] Generic M3 in QEMU (MPS2-AN385)
├─ qemu-cortex-m4f/        [MOD] Generic M4F in QEMU (MPS2-AN386)
├─ qemu-cortex-m7f/        [MOD] Generic M7F in QEMU (MPS2-AN500)
└─ raspberrypi-pico/       [LEG] Upstream single-core RP2040 platform
```

Scheduling Modes Supported

1. **SMP n:** Kernel built with `OS_USE_SMP_SCHEDULER`, `OS_NCPU = n` (e.g., 2 on RP2350 / AN521, 3 on Lyra, 4 on BCM2837 and native).

2. **SMP n, 1 active:** Multi-core image where secondary cores remain parked in spin loops.
3. **SMP x1:** SMP scheduler algorithm running on single-core hardware (`OS_NCPU = 1`).
4. **Single-core:** Upstream kernel execution without SMP scheduler infrastructure.
5. **Bare-metal:** Board diagnostic probe applications executing with no kernel scheduler (`BOARD_TEST_NO_KERNEL`).

5. Execution Harness, Runners & Verdict Protocol (`test_smp/`)

5.1 The `test_smp/` Runner Layer

The `smp` branch centralizes all test execution logic in `test_smp/`:

- `run-qemu.sh` :
 - Launches QEMU with architecture-neutral arguments.
 - Enforces per-test execution timeout budgets via internal `timeout_for()` lookup.
 - Automatically creates and tears down a **4 GiB raw sparse SD card image** for filesystem test suites (`sd_test` , `smp-mat-sdcard-test`).
 - Employs an AArch64-to-AArch32 transition boot shim (`shim8.img`) to execute AArch32 binaries on QEMU's `raspi3b` machine at entry `0x10000` .
- `run-host.sh` :
 - Mirrors `run-qemu.sh` for POSIX native tests.
 - Manages per-test logging into `.host-logs/<app>.log` and supports isolated execution via `UOS_RUN_ONLY=<app>` .
- `run-hw.sh` :
 - Hardware flash runner driven entirely through **OpenOCD** (no GDB, no serial port capture).
 - Leaves the physical UART interface free for user monitoring via `picocom` / `tio` .
 - Zeroes the secondary core spin table (`__smp_spin[]`) before starting execution.
 - Enforces the rule: **one test per power cycle** on development boards without system reset (SRST).

5.2 Verdict Protocol & Termination Contract

Tests in `xpack-development` signaled completion via the main function return code or semihosting exit. The `smp` branch establishes a standardized dual-channel contract:

1. **Greppable Verdict String:** The test application prints a definitive terminal line: `text RESULT: PASS`
`RESULT: FAIL RESULT: SKIP (<reason>)`
2. **Explicit Stop via `hw_result.hpp`:** `cpp if (all_ok) { write_str ("RESULT: PASS\n"); hw_result::ok (); // Issues semihosting SYS_EXIT success } else { write_str ("RESULT: FAIL\n"); hw_result::fail (); // Issues semihosting SYS_EXIT failure }` This prevents hardware boards from continuing into unhandled idle loops when monitored by automated probes.

3. **Compile-Time Board Invariants via `board-contract.cpp`**: Ensures any new board explicitly defines required properties (`PORT_RAM_BASE`, `PORT_RAM_END`, `OS_NCPU`, `PORT_GREETING`, `SEMIHOST_TRAP_CHOSEN`). Missing definitions trigger loud compile errors instead of silently inheriting incorrect defaults from previous targets.

5.3 CTest Filtering and Naming Conventions

- **CTest Names:** Structured as `<platform>-<app>-<variant>`, where variant is `-qemu`, `-host`, or `-hwd` (e.g., `aarch32-rpi-zero-2w-smp_test0-qemu`).
- **Hardware Exclusion:** Hardware cases are tagged with `LABELS "hwd"`. Standard test executions invoke: `sh ctest -V -LE hwd` preventing CI or emulator runs from stalling on physical probe requirements.

6. Manifest & Configuration Matrix (`tests/package.json`)

The test manifest `tests/package.json` expands substantially (over 900 new lines) to accommodate the multi-core matrix:

6.1 Action Hierarchy

- **xpack-development**: `json "test-all": [ "xpm run test-native-cmake", "xpm run test-cortex-cmake" ]`
- **smp**: `json "test-all": [ "xpm run test-native-cmake", "xpm run test-cortex-cmake", "xpm run test-smp-cmake" ]` where `"test-smp-cmake"` orchestrates:
 - `test-aarch32-rpi-zero-2w-cmake`
 - `test-aarch32-rpi3b-cmake`
 - `test-aarch64-rpi-zero-2w-cmake`
 - `test-aarch64-rpi3b-cmake`
 - `test-2xcortex-m33-cmake`
 - `test-pico2-1cpu-cmake`
 - `test-cortexm-pico2-cmake`
 - `test-cortexm-pico2-rp2350b-psram-cmake`

6.2 Per-Test Actions for IDE Integration

Every individual test case across each platform receives a dedicated action:

```
"test-smp-mat-test-qemu": "{ { properties.commandCMakePrepareWithToolchain } } &&
{ { properties.commandCMakeBuild } } && cd { { properties.buildFolderRelativePath } } && ctest -V -R
aarch32-rpi-zero-2w-smp-mat-test-qemu"
```

This enables direct discovery and invocation inside the VS Code xPack extension.

6.3 Toolchain Modernization

- Pinned GNU Arm Embedded Toolchain upgraded to `arm-none-eabi-gcc 15.2`.
 - Added mixin `clang-gcc14-properties`: allows Clang 16–18 builds on native Linux to bind against xPack GCC 14 `libstdc++` via `--gcc-toolchain`, preventing syntax errors caused by older Clang frontends parsing newer host GCC standard library headers.
-

7. Device Emulation Plumbing (`tests/device-qemu-cortexm/`)

To support Cortex-M33 dual-core emulation in QEMU, the shared QEMU device support package is enhanced:

1. Linker Scripts for SSE-200:

2. `mem-mps2-an505.ld` (single M33) and `mem-mps2-an521.ld` (dual M33).
3. Maps 4 MiB flash at `0x10000000` and 16 MiB flat system RAM at `0x80000000` (bypassing the limited 128 KiB internal SRAM).

4. ARMv8-M Vector Table Extensions (`vectors-cortexm.c`):

5. Added `SecureFault_Handler` for ARMv8-M baseline security exceptions.
6. Registered SSE-200 Message Handling Unit (MHU) interrupts:
 - IRQ 6: `MHU0_IRQHandler` (core 1 reschedule IPI).
 - IRQ 7: `MHU1_IRQHandler` (core 0 reschedule IPI).

7. Multi-Core SysTick Separation (`exception-handlers.cpp`):

8. The generic `SysTick_Handler` is conditionally omitted when `OS_USE_SMP_SCHEDULER` is defined. This allows the Cortex-M33 SMP port to supply its own per-core timer handler where only core 0 advances RTOS tick counts while secondary cores execute context switching without advancing system time.
-

8. Summary Comparison Matrix

Feature	xpack-development	smp
Build System Orchestrator	CMake + xpm + CTest	CMake + xpm + CTest + test_smpl
Sibling Port Dependencies	None (in-tree library)	posix-arch, cortexm, aarch32, aarch64
Max Active Cores per Test	1	4
Number of Platforms	9	22
Test Registration	Explicit in CMakeLists.txt	Dynamic filesystem globbing in ports
Preemption & FPU Verification	Basic mutex stress	Dedicated fp-switch test suite with core migration checks
Semi-hosting Execution Control	Process return codes	Machine-greppable RESULT: + hw_result::ok()/fail()
Hardware Probe Automation	Manual OpenOCD / GDB	Pure OpenOCD run-hw.sh, one test per power cycle
Filesystem Test Emulation	Host only	Automatic 4 GiB sparse SD card image creation & cleanup
Root CMake Library Target	Monolithic micro-os-plus::iii	Modular micro-os-plus::iii core + optional groups

9. Transition Pathways: Rebuilding the SMP Test Framework from xpack-development

The complete roadmap for lifting the xpack-development branch to the smp branch is documented in <docs/smp-integration/xpack-dev-smp.md>. Replicating the SMP test framework is not a monolithic operation, but the culmination of a staged migration pipeline designed to ensure continuous verification across all platforms.

S0: Pre-Migration Subtree Dissolution of Legacy `devices`	
(Absorb drivers into posix-arch, cortexm, aarch32, aarch64; retire repo)	
S1: Single-Core Defect Corrections & Hardening (PR #1 – PR #22 / K01-K29)	
(Continuous G1 Gate: 72/72 configs pass on xpack-development harness)	
S2: Core Multi-Core SMP Kernel Infrastructure (K30-K38, P05-P11, C05-C07)	
(Strictly guarded behind OS_USE_SMP_SCHEDULER; Continuous G1 & G2 Gates)	
S3: Architecture Ports & Silicon Drivers (C08-C12, A32-04..12, A64-04..11)	
(Generic M33, RP2350, AArch32 & AArch64 bare-metal boot and ports)	
S4: Test Harness & Execution Modernization (K40-K63)	
└─ S4a: Build & Harness Infrastructure (K40-K48)	
└─ S4b: New Test Suite: fp-switch (K49)	
└─ S4c: Platform Matrix Activation (K50-K63; Gate G3 per platform)	
S5 / S6: Documentation, Final Parity & Full Matrix Acceptance (G-final)	

9.1 Stage S4: The Core Test Framework Lifting Sequence

Stage S4 in [xpack-dev-smp.md](#) details the specific commits and files required to transform the test framework:

1. Stage S4a: Build and Harness Infrastructure (Commits K40–K48)

- Cross-Toolchain Profiles (K40):** Add CMake toolchain configurations for `arm-none-eabi`, `aarch64-none-elf`, and native environments in `cmake/toolchains/*.cmake`.
- Unified Application Builder (K41):** Introduce `cmake/uos-app.cmake` supplying `uos_add_app()` and `uos_add_test_app()` to compile port applications against the kernel and drivers with a single invocation.
- Modular Kernel Targets (K42):** Refactor root `CMakeLists.txt` into granular interface libraries (`micro-os-plus::iii-core`, `micro-os-plus::iii-posix-io`, `micro-os-plus::iii-startup`, `micro-os-plus::port-smp-decls`, `micro-os-plus::test-support`), while preserving `micro-os-plus::iii` as the umbrella alias.
- The Shared Verdict Contract (K43a):** Introduce `test_smpl/include/hw_result.hpp` and `test_smpl/src/board-contract.cpp` to standardize pass/fail semantics and compile-time board checks.
- Execution Runners (K43b – K43d):** Add `test_smpl/run-host.sh`, `test_smpl/run-qemu.sh`, and `test_smpl/run-hw.sh`.

6. **Dynamic Sibling Port Dispatch (K44)**: Upgrade `tests/cmake/tests-main.cmake` to detect sibling port directories via `UOS_<PORT>_DIR` based on `PLATFORM_NAME`.
7. **Native Port Test Integration (K45)**: Update `tests/platforms/native/CMakeLists.txt` to glob and run `posix-arch` tests.
8. **QEMU Cortex-M Realignment (K46a – K46d)**: Retarget single-core QEMU platforms (`qemu-cortex-m0`, `m3`, `m4f`, `m7f`) to link against the Cortex-M port generic QEMU cores.
9. **QEMU ARMv8-M Emulation Plumbing (K47)**: Add MPS2 AN505/AN521 linker scripts and extend `vectors-cortexm.c` with SSE-200 MHU inter-processor interrupt handlers.
10. **Nucleo Realignment (K48)**: Update `nucleo-f411re` to link against the dissolved `soc-stm32f411xe` target.

2. Stage S4b: New Test Suite (Commit K49)

- Add `tests/sources/fp-switch/` to validate preemption and core-migration integrity of hardware floating-point registers.

3. Stage S4c: Platform Matrix Activation (Commits K50–K63)

- Incrementally add each new platform folder in `tests/platforms/` alongside its debug and release build configurations in `tests/package.json`:
- **K50** : `2xcortex-m33` (QEMU dual M33)
- **K51** : `pico2-1cpu` (QEMU single M33)
- **K52 – K54** : RP2350 Pico 2 physical boards (`cortexm-pico2`, `cortexm-pico2-rp2350b-psram`, `cortexm-pico2-pizero`)
- **K55 – K57** : STM32 physical boards (`cortexm-nucleof411`, `cortexm-weactf411`, `cortexm-weactf412`)
- **K58 – K59** : `aarch32-rpi-zero-2w` and `aarch32-rpi3b`
- **K60 – K61** : `aarch64-rpi-zero-2w` and `aarch64-rpi3b`
- **K62** : `aarch32-luckfox-lyra`
- **K63** : Debug/Release aliases and Clang 13–15 configurations in `package.json`.

9.2 Three Implementation Strategies

Depending on whether the goal is an upstream-ready pull request series, an accelerated test modernization, or an evaluation branch, three distinct pathways exist:

Strategy A: The Canonical Upstream-Grade Staged PR Sequence

- **Concept**: Strictly follow the progressive PR runbook in `xpack-dev-smp.md`.
- **Execution**:
 - Complete S0 (dissolving `devices` without breaking single-core builds).
 - Complete S1 (landing bugfixes and hardening with Gate G1: 72/72 passing configurations).

- Land S2 (kernel SMP guarded by `OS_USE_SMP_SCHEDULER`) and S3 (port implementations).
- Implement S4 commits atomically, validating Gate G3 after each platform is introduced.
- **Pros:** Clean bisectability, zero regressions on existing upstream platforms, upstream review-ready commits.
- **Cons:** Requires completing all prior kernel and port prerequisites before the full matrix can be exercised.

Strategy B: Harness-First Staged Architecture (Accelerated Modernization)

- **Concept:** Land the harness infrastructure changes (Stage S4a/S4b) ahead of full multi-core silicon port implementations, using fallback shims.
- **Execution:**
- **Refactor root `CMakeLists.txt`:** Split monolithic kernel into `micro-os-plus::iii-core` and optional interface libraries.
- **Introduce `test_smpl/`:** Add `hw_result.hpp`, `run-qemu.sh`, and `run-host.sh`.
- **Wire Dynamic Dispatch with Upstream Fallbacks:** Update `tests-main.cmake` so that any unmigrated platform falls back to `add_subdirectory("../top-bin")`.
- **Run Suites on Host & Single-Core QEMU:** Run `rtos-apis`, `mutex-stress`, `cmsis-os-validator`, and `fp-switch` on the existing single-core QEMU and host configurations.
- **Progressively Attach Architecture Ports:** As each architecture port repository matures, add its platform entry into `tests/platforms/` and `tests/package.json`.
- **Pros:** Harness improvements (structured logging, timeouts, clean verdicts) become immediately available on `xpack-development` without waiting for physical board ports.
- **Cons:** Requires temporary backward-compatibility branches in `tests-main.cmake`.

Strategy C: Direct Integration via Branch Replay / Cherry-Pick

- **Concept:** Rapidly achieve full SMP test harness capabilities on an integration branch by cherry-picking the S4 commit series directly from the `smp` branch.
- **Execution:**
- Ensure the workspace has the sibling port repositories cloned on their respective `smp` branches (`micro-os-plus-iii-cortexm`, `micro-os-plus-iii-posix-arch`, `micro-os-plus-iii-aarch32`, `micro-os-plus-iii-aarch64`).
- Cherry-pick the modular target split from root `CMakeLists.txt` (commit `K42`).
- Cherry-pick the harness commits `K43a` through `K63`.
- Run `xpm run install-all` followed by `xpm run test-all`.
- **Pros:** Fastest route to establishing an identical test environment for verification or port development.
- **Cons:** Bypasses intermediate regression checkpoints; non-trivial conflict resolution if kernel source divergence exists.

9.3 Verification Gates & Acceptance Criteria

Every transition step between `xpack-development` and `smp` must satisfy strict gate criteria executed from the `tests/` directory:

Gate	Command	Acceptance Criterion
G0 (Post-S0)	<code>xpm run test-native-cmake && xpm run test-cortex-cmake</code>	Identical baseline pass status as <code>xpack-development</code> with <code>devices</code> dissolved.
G1 (Post-S1 / S4a)	<code>xpm run test-native-cmake && xpm run test-cortex-cmake</code>	All 24 configurations × 3 test suites pass (72/72 green).
G2 (Post-S2)	<code>xpm run test-native-cmake</code>	Host SMP tests pass cleanly with host threads acting as CPUs.
G3 (Per Platform)	<code>xpm run test-<platform>-cmake</code>	All non-hardware cases pass (<code>ctest -V -LE hwd</code>).
G-final (Full Parity)	<code>xpm run test-all</code>	Full matrix green across <code>test-native-cmake</code> , <code>test-cortex-cmake</code> , and <code>test-smp-cmake</code> .